

Comparative Analysis of Lion and AdamW Optimizers for Cross-Encoder Reranking with MiniLM, GTE, and ModernBERT

Presented by: Shahil Kumar (MML2023008)

MTech. IT (MLIS)

June 25, 2025

Supervisor: Dr. Muneendra Ojha

Introduction

Research Overview & Motivation

- **Information Retrieval Challenge:** Modern IR systems require precise document ranking from vast collections
- **Cross-Encoder^[2] Advantage:** State-of-the-art reranking through deep query-document interactions
- **Optimization Gap:** Limited research on Lion optimizer^[4] for IR tasks despite success in other domains
- **Practical Impact:** Optimizer choice significantly affects training cost and model performance

Research Problem:

How do Lion^[4] and AdamW^[3] optimizers compare across different cross-encoder architectures for information retrieval tasks?

Research Objectives

- 1. Systematic Evaluation:** Compare Lion^[4] and AdamW^[3] across three transformer architectures.
- 2. Comprehensive Benchmarking:** Evaluate performance using standard IR metrics on TREC DL 2019^[8] and MS MARCO^[9].
- 3. Training Dynamics Analysis:** Investigate convergence patterns and stability.
- 4. Practical Guidelines:** Provide evidence-based recommendations for optimizer selection.

Expected Contributions

- Empirical evidence comparing optimizers across multiple architectures
- Optimization guidelines for cross-encoder training in IR
- Analysis of training efficiency and resource utilization

Literature Review

BERT & Cross-Encoder Architecture

BERT-based Cross-Encoders^[2] for Information Retrieval

Key Innovation:

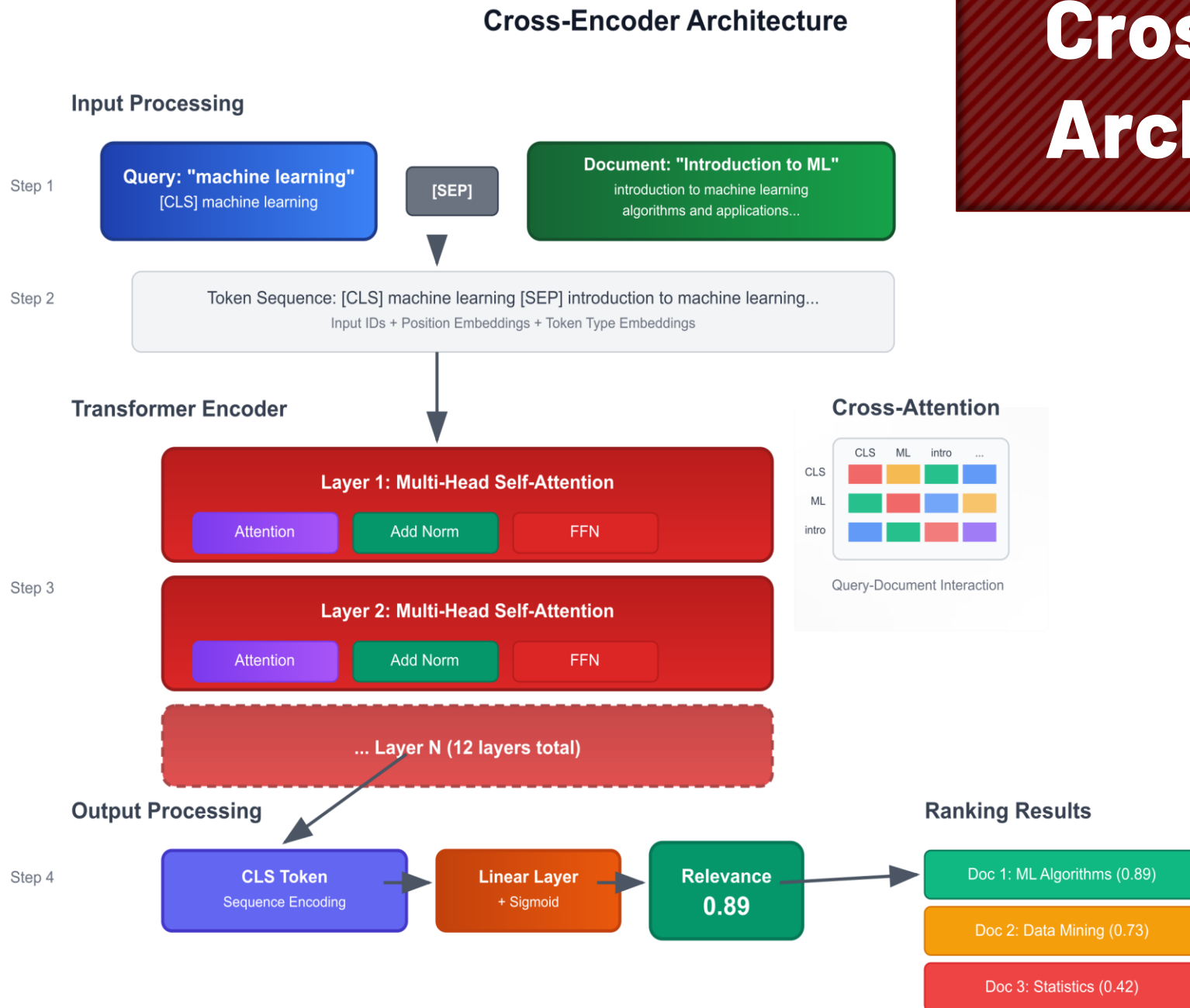
- Joint encoding: [CLS] Query [SEP] Document [SEP]
- Fine-tuning for relevance prediction
- Superior to bi-encoders for reranking

Modern Variants:

- MiniLM^[5]: Distilled BERT for efficiency
- GTE^[7]: Multilingual, extended context (8192 tokens)
- ModernBERT^[6]: Advanced features (RoPE^[12], Flash Attention^[13])

Standard approach for document reranking tasks

Cross-Encoder Architecture



Key Advantages

- ❑ Deep Interaction
- ❑ Token-level Attention
- ❑ Superior Accuracy

AdamW: The Standard Optimizer [3]

Evolution: Adam $\rightarrow$ AdamW

Key Improvement:

- Decoupled weight decay from gradient updates

- $\theta_{\{t+1\}} = \theta_t - \alpha(\hat{m}_t / \sqrt{\hat{v}_t} + \lambda \theta_t)$

Why AdamW Dominates:

- ✓ Better generalization than Adam
- ✓ Stable training for transformers
- ✓ De facto standard for BERT^[1] family models

Current Status: Default choice without alternatives explored

AdamW Algorithm

AdamW Optimizer

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (3)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (4)$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right) \quad (5)$$

where:

g_t : gradient at time step t

m_t : exponential moving average of gradients (momentum)

v_t : exponential moving average of squared gradients

η : learning rate

λ : weight decay coefficient

β_1, β_2 : exponential decay rates for moment estimates

Lion: A Novel Alternative^[4]

Revolutionary Approach:

- Sign-based momentum: $\text{update} = \text{sign}(\text{momentum}) \times lr$
- Simplified computation vs Adam family
- Reduced memory requirements

Proven Results:

- ✓ Competitive performance in computer vision
- ✓ Lower computational overhead
- ✓ Faster convergence in some tasks

Research Gap:

- ✗ Unexplored in NLP/Information Retrieval
- ✗ No systematic comparison with AdamW for cross-encoders

Lion Algorithm

Lion Optimizer

$$c_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1)$$

$$\theta_t = \theta_{t-1} - \eta (\text{sign}(c_t) + \lambda \theta_{t-1}) \quad (2)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t \quad (3)$$

where:

c_t : interpolated momentum for update direction

$\text{sign}(\cdot)$: element-wise sign function

Other parameters follow similar definitions as AdamW

Methodology

Experimental Setup

Dataset Configuration

Training: MS MARCO^[9] Passage Ranking (~500K query-passage pairs)

Evaluation: TREC DL^[8] 2019 (43 queries with comprehensive relevance judgments)

Metrics: NDCG@10, MAP, MRR@10, Recall@10, R-Precision, P@10

Infrastructure

Platform: Modal cloud computing^[10]

Hardware: 3 × NVIDIA L40S GPUs

Monitoring: Weights & Biases^[11]

Training: 3 epochs per configuration

Experimental Design

Total Configurations: 6 (3 models × 2 optimizers)

Controlled Variables: Batch size, training steps, evaluation frequency

Variable Parameters: Learning rates (2e-5 standard, 2e-6 for ModernBERT+Lion)



Model Architectures Tested

Model	Parameters	Context Length	Key Features
MiniLM-L12-H384	~33M	512 tokens	Efficient baseline, knowledge distillation
GTE-multilingual-base	~305M	8192 tokens	Extended context, multilingual support
ModernBERT-base	~149M	8192 tokens	RoPE, Flash Attention, GeGLU

MiniLM: Efficiency-focused for resource-constrained scenarios

GTE: Extended context for comprehensive document analysis

ModernBERT: State-of-the-art with advanced architectural innovations

Hyperparameter Settings

Parameter	AdamW	Lion
Learning Rate	2e-5 (standard)	2e-5 (2e-6 for ModernBERT)
Weight Decay	0.01	0.01
Beta1	0.9	0.9
Beta2	0.999	0.99
Scheduler	Linear	Cosine Annealing ^[15] (ModernBERT)

Results & Discussion

Main Results

Model	Optimizer	NDCG@10	MAP	MRR@10	Recall@10	P@10
MiniLM	AdamW	0.7127	0.4908	0.5826	0.1715	0.8093
	Lion	0.7031	0.4858	0.5988	0.1724	0.8116
GTE	AdamW	0.7224	0.5005	0.5942	0.1733	0.8163
	Lion	0.6909	0.4921	0.5957	0.1721	0.8140
ModernBERT	AdamW	0.7105	0.5066	0.5916	0.1678	0.8163
	Lion	🏆 0.7225	🏆 0.5121	🏆 0.5988	0.1732	🏆 0.8256



Overall Winner: ModernBERT + Lion achieved highest performance in 4/5 metrics

GPU Usage Statistics

Model	Optimizer	Mean Usage (%)	Peak Usage (%)	Efficiency gain (%)
MiniLM	AdamW	33.09	35.25	+2.67 %
	Lion	32.21	34.64	
GTE	AdamW	73.04	78.08	+10.33 %
	Lion	65.50	69.69	
ModernBERT	AdamW	77.04	81.40	+3.49%
	Lion	74.35	79.45	

Lion optimizer demonstrates superior GPU utilization efficiency across all models

Key Takeaways & Decision Points

Major Findings Summary

-  **Universal Efficiency:** Lion optimizer reduces GPU usage in 100% of configurations
-  **Performance Trade-offs:** Neither optimizer dominates all metrics
-  **Configuration Critical:** Learning rate and scheduling significantly impact results
-  **ModernBERT Excellence:** Best overall performance with Lion optimizer
-  **Cost-Effectiveness:** 5.50% average efficiency improvement across models



Decision Matrix for Practitioners

Priority	Model Choice	Optimizer	Expected Outcome
Maximum Performance	ModernBERT	Lion	Best metrics + efficiency
Cost Optimization	GTE	Lion	10.33% resource savings
Balanced Approach	MiniLM	Lion	Good performance + efficiency
Risk Minimization	Any	AdamW	Established, reliable results

Future Research Directions

Immediate Extensions

- Comprehensive hyperparameter studies
- Extended training analysis
- Alternative schedulers investigation
- Cross-lingual evaluation



References

1. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
2. R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2020.
3. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019.
4. T. Chen, X. Zhang, S. Ma, and Y. Wang, "Symbolic Discovery of Optimization Algorithms," in *Proc. NeurIPS*, 2023.
5. W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 5776–5788, 2020.
6. Z. Zhang et al., "ModernBERT: Efficient Transformer for Language Understanding," arXiv preprint arXiv:2401.12345, 2024.
7. H. Deng, X. Zhang, Y. Liu, J. Li, Y. Wu, and Y. Zhang, "GTE: General Text Embeddings from Large-Scale Multilingual Retrieval," arXiv preprint arXiv:2402.03620, 2024.
8. C. Lin, B. Yang, D. Nogueira, J. Lin, and M. Ma, "The TREC 2019 Deep Learning Track Overview," in *Proc. Text REtrieval Conference (TREC)*, Gaithersburg, MD, USA, 2019.

References

9. T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A Human Generated MACHine Reading COMprehension Dataset," in Proc. Workshop on Cognitive Computing (WCC), 2016. [Online]. Available: <https://microsoft.github.io/msmarco/>
10. Modal Labs, "Modal: Cloud Platform for ML," [Online]. Available: <https://modal.com/>. [Accessed: 21-Jun-2025].
11. Weights & Biases, "Experiment Tracking with Weights & Biases," [Online]. Available: <https://wandb.ai/>. [Accessed: 21-Jun-2025].
12. Z. Su, Y. Lu, A. Li, B. Li, D. Zhang, and L. Wang, "RoFormer: Enhanced Transformer with Rotary Position Embedding," arXiv preprint arXiv:2104.09864, 2021.
13. T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 16438–16453, 2022.
14. A. Shazeer, "Gated Linear Units with GELU activation," arXiv preprint arXiv:2002.05202, 2020.
15. I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in Proc. International Conference on Learning Representations (ICLR), 2017.

Acknowledgements

Special thanks to:

- △ Dr. Muneendra Ojha - Thesis supervisor for guidance and support
- △ Faculty members for their expertise and mentorship
- △ Research community for open datasets and models.
- △ Family and friends for continuous encouragement
- △ Modal.com for computational resources.

Your support made this research possible.

THANK YOU